Trabajo Práctico — Programación IV

Aplicación Web de Gestión de Usuarios

Universidad Tecnológica Nacional
Facultad Regional [Tu Facultad]
Materia: Programación IV

Alumno: [Tu Nombre]
Legajo: [Tu Legajo]
Docente: [Nombre del Docente]
Fecha: Septiembre 2026

---

1. Introducción

El presente trabajo práctico consiste en el desarrollo de una aplicación web para la gestión de usuarios con autenticación y roles simples (docente/administrador y alumno). La aplicación fue desarrollada utilizando Laravel 12 como framework PHP, MySQL como sistema de gestión de base de datos, y TailwindCSS para los estilos del frontend.

El objetivo principal es demostrar los conocimientos adquiridos en materia de desarrollo web con PHP, aplicando buenas prácticas de arquitectura, seguridad y diseño de interfaces.

---

2. Requerimientos Funcionales

2.1 Autenticación y Registro

Campo	Tipo	Obligatorio
Nombre	String	Sí
Email	Email	Sí

Contraseña	String (min 8)	Sí
Teléfono	String	No
URL Profesional	URL	No
Foto de perfil	Imagen (jpg, png, webp)	Sí

2.2 Roles

- **Administrador (docente):** Puede ver el listado y detalle de todos los usuarios registrados.
- **Alumno:** Solo puede ver y editar su propio perfil.

2.3 Perfil de Usuario

- Campo de teléfono con hipervínculo directo a WhatsApp.
- Campo de enlace a red profesional que abra en nueva pestaña.
- Foto de perfil visible en perfil y listado.

---

3. Requerimientos Técnicos

Componente	Tecnología
Framework	Laravel 12 (v13.32.0 instalado)
Base de datos	MySQL 8.0
Frontend	Blade + TailwindCSS
Autenticación	Laravel Breeze
Servidor PHP	PHP 8.3.7

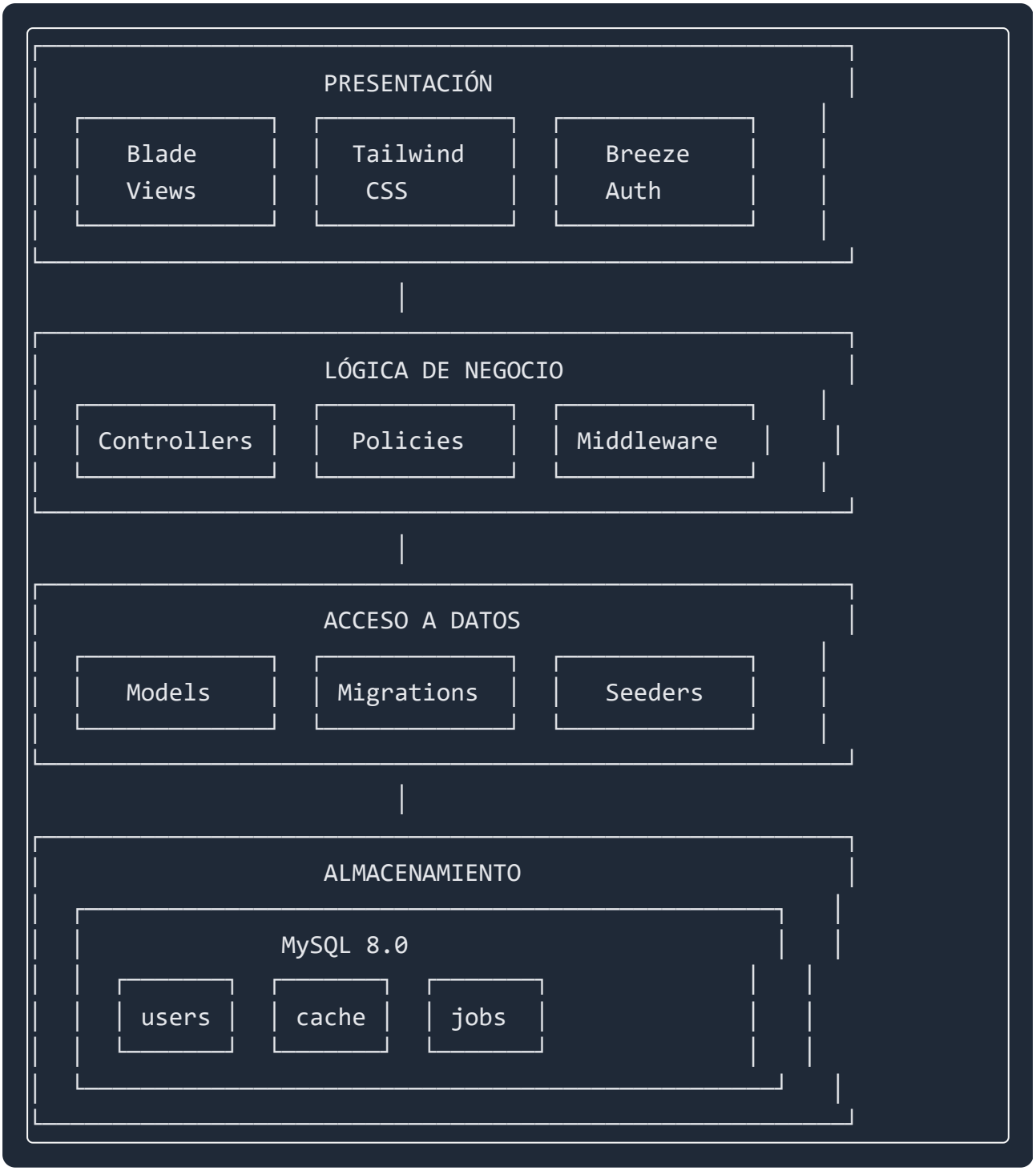
---

4. Arquitectura del Sistema

4.1 Estructura de Directorios

```
lia-app/
├── app/
│   ├── Http/
│   │   ├── Controllers/
│   │   │   ├── Admin/
│   │   │   │   └── UserController.php
│   │   │   ├── Auth/
│   │   │   │   └── RegisteredUserController.php
│   │   │   └── ProfileController.php
│   │   └── Middleware/
│   │       └── AdminMiddleware.php
│   ├── Models/
│   │   └── User.php
│   └── Policies/
│       └── UserPolicy.php
├── database/
│   ├── migrations/
│   │   ├── 0001_01_01_000000_create_users_table.php
│   │   └── 2026_09_16_173215_add_custom_fields_to_users_table.php
│   └── seeders/
│       └── DatabaseSeeder.php
├── resources/views/
│   ├── admin/users/
│   │   ├── index.blade.php
│   │   ├── create.blade.php
│   │   ├── edit.blade.php
│   │   └── show.blade.php
│   ├── auth/register.blade.php
│   ├── profile/
│   │   ├── show.blade.php
│   │   ├── edit.blade.php
│   │   └── partials/
│   │       └── update-profile-information-form.blade.php
│   └── dashboard.blade.php
├── routes/
│   └── web.php
└── config/services.php
```

4.2 Diagrama de Componentes



---

5. Diseño de Base de Datos

5.1 Tabla users

Campo	Tipo	Descripción
id	bigint (PK)	Identificador único
name	varchar(255)	Nombre del usuario
email	varchar(255)	Email único
is_admin	boolean	Rol: true=Admin, false=Alumno
phone	varchar(20)	Teléfono (opcional)
professional_url	varchar(255)	URL red profesional (opcional)
photo_path	varchar(255)	Ruta foto de perfil
password	varchar(255)	Contraseña hasheada
email_verified_at	timestamp	Verificación de email
remember_token	varchar(100)	Token "recordarme"
created_at	timestamp	Fecha de creación
updated_at	timestamp	Última actualización

5.2 Migraciones

Migración 1: Tabla users base

```
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('email')->unique();
    $table->timestamp('email_verified_at')->nullable();
    $table->string('password');
    $table->rememberToken();
    $table->timestamps();
});
```

Migración 2: Campos personalizados

```
Schema::table('users', function (Blueprint $table) {
    $table->boolean('is_admin')->default(false)->after('email');
    $table->string('phone')->nullable()->after('is_admin');
    $table->string('professional_url')->nullable()-
>after('phone');
    $table->string('photo_path')->nullable()-
>after('professional_url');
});
```

---

6. Implementación

6.1 Modelo User

```php
class User extends Authenticatable
{
    #[Fillable(['name', 'email', 'password', 'phone',
                'professional_url', 'photo_path', 'is_admin'])]
    #[Hidden(['password', 'remember_token'])]

    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
            'is_admin' => 'boolean',
        ];
    }

    public function getPhotoUrlAttribute(): string
    {
        return $this->photo_path
            ? asset('storage/' . $this->photo_path)
            : 'https://ui-avatars.com/api/?name='
               . urlencode($this->name);
    }

    public function getWhatsAppUrlAttribute(): string
    {
        $prefix = config('services.whatsapp.prefix', '54');
        $phone = preg_replace('/[^0-9]/', '', $this->phone);
        return "https://wa.me/{$prefix}{$phone}";
    }
}
```

6.2 Middleware de Administrador

```php
class AdminMiddleware
{
    public function handle(Request $request, Closure $next):
Response
    {
        if (!$request->user() || !$request->user()->is_admin) {
            abort(403, 'No tienes permiso para acceder.');
        }
        return $next($request);
    }
}
```

6.3 Policy de Usuarios

```php
class UserPolicy
{
    public function viewAny(User $user): bool
    {
        return $user->is_admin;
    }

    public function view(User $user, User $model): bool
    {
        return $user->is_admin || $user->id === $model->id;
    }

    public function update(User $user, User $model): bool
    {
        return $user->is_admin || $user->id === $model->id;
    }

    public function delete(User $user, User $model): bool
    {
        return $user->is_admin;
    }
}
```

6.4 Rutas

```
// Rutas protegidas por auth
Route::middleware('auth')->group(function () {
    Route::get('/profile', [ProfileController::class, 'show']);
    Route::get('/profile/edit', [ProfileController::class,
'edit']);
    Route::patch('/profile', [ProfileController::class,
'update']);
    Route::delete('/profile', [ProfileController::class,
'destroy']);
});

// Rutas admin (auth + admin middleware)
Route::middleware(['auth', 'admin'])
     ->prefix('admin')->name('admin.')->group(function () {
    Route::get('/users', [UserController::class, 'index']);
    Route::get('/users/create', [UserController::class,
'create']);
    Route::post('/users', [UserController::class, 'store']);
    Route::get('/users/{user}', [UserController::class, 'show']);
    Route::get('/users/{user}/edit', [UserController::class,
'edit']);
    Route::put('/users/{user}', [UserController::class,
'update']);
    Route::delete('/users/{user}', [UserController::class,
'destroy']);
});
```

6.5 Controlador de Registro

```php
public function store(Request $request): RedirectResponse
{
    $request->validate([
        'name' => ['required', 'string', 'max:255'],
        'email' => ['required', 'email', 'unique:users'],
        'password' => ['required', 'confirmed', 'min:8'],
        'phone' => ['nullable', 'string', 'max:20'],
        'professional_url' => ['nullable', 'url'],
        'photo' => ['required', 'image',
'mimes:jpg,jpeg,png,webp'],
    ]);

    $photoPath = $request->file('photo')
                         ->store('photos', 'public');

    $user = User::create([
        'name' => $request->name,
        'email' => $request->email,
        'password' => Hash::make($request->password),
        'phone' => $request->phone,
        'professional_url' => $request->professional_url,
        'photo_path' => $photoPath,
    ]);

    Auth::login($user);
    return redirect('/dashboard');
}
```

---

7. Seguridad Implementada

Medida	Implementación
Autenticación	Laravel Breeze con sesiones
Autorización	Policies + Middleware admin
Validación	FormRequest y validación en controladores

Protección CSRF	Token CSRF en todos los formularios
Hash de contraseñas	bcrypt via Eloquent casts
Almacenamiento seguro	Fotos en storage/ con enlace simbólico
Configuración WhatsApp	Prefijo configurable en .env

---

8. Guía de Instalación

8.1 Requisitos Previos

- PHP 8.3 o superior
- Composer 2.x
- MySQL 8.0
- Node.js (para compilar assets)

8.2 Pasos de Instalación

```
# 1. Clonar el repositorio
git clone [URL_DEL_REPOSITORIO]
cd lia-app

# 2. Instalar dependencias PHP
composer install

# 3. Instalar dependencias Node
npm install

# 4. Configurar entorno
cp .env.example .env
php artisan key:generate

# 5. Configurar base de datos en .env
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=lia_tp
DB_USERNAME=root
DB_PASSWORD=

# 6. Crear base de datos
mysql -u root -e "CREATE DATABASE lia_tp CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;"

# 7. Ejecutar migraciones y seeders
php artisan migrate:fresh --seed

# 8. Crear enlace de almacenamiento
php artisan storage:link

# 9. Compilar assets
npm run build

# 10. Iniciar servidor
php artisan serve
```

8.3 Usuarios de Prueba

Rol	Email	Contraseña
Administrador	admin@lia.com	password
Alumno	lia@lia.com	password

---

9. Guía de Uso

9.1 Flujo del Administrador

1. Acceder a `http://localhost:8000/login`
2. Iniciar sesión con `admin@lia.com` / `password`
3. Acceder a `/admin/users` para ver listado de usuarios
4. Crear, editar o eliminar usuarios desde el panel

9.2 Flujo del Alumno

1. Acceder a `http://localhost:8000/register`
2. Completar formulario con foto de perfil obligatoria
3. Iniciar sesión automáticamente
4. Ver perfil en `/profile`
5. Editar perfil en `/profile/edit`

---

10. Conclusiones

El desarrollo de esta aplicación permitió aplicar los conocimientos adquiridos en materia de Programación IV, incluyendo:

- **Arquitectura MVC:** Separación clara de modelos, vistas y controladores.
- **Autenticación y autorización:** Uso de Breeze, Policies y Middlewares.
- **Validación de datos:** FormRequest y validación en controladores.
- **Interacción con base de datos:** Migraciones, Eloquent ORM y Seeders.
- **Diseño responsive:** TailwindCSS para estilos modernos y adaptables.

- **Seguridad:** Protección CSRF, hash de contraseñas, control de acceso.

El framework Laravel 12 demostró ser una herramienta robusta y eficiente para el desarrollo de aplicaciones web, facilitando la implementación de funcionalidades complejas con código limpio y mantenible.

---

11. Anexo

11.1 Estructura de Rutas

Método	Ruta	Descripción
GET	/	Página principal
GET	/dashboard	Dashboard del usuario
GET	/login	Formulario de login
POST	/login	Autenticar usuario
GET	/register	Formulario de registro
POST	/register	Registrar usuario
GET	/profile	Ver perfil propio
GET	/profile/edit	Editar perfil
PATCH	/profile	Actualizar perfil
DELETE	/profile	Eliminar cuenta
GET	/admin/users	Listado de usuarios (admin)
GET	/admin/users/create	Crear usuario (admin)
POST	/admin/users	Guardar usuario (admin)
GET	/admin/users/{id}	Ver usuario (admin)
GET	/admin/users/{id}/edit	Editar usuario (admin)

PUT	/admin/users/{id}	Actualizar usuario (admin)
DELETE	/admin/users/{id}	Eliminar usuario (admin)

11.2 Configuración WhatsApp

El prefijo de país se configura en el archivo `.env`:

```
WHATSAPP_COUNTRY_PREFIX=54
```

Documento generado para la materia Programación IV
Universidad Tecnológica Nacional